🔒 Docker Step 5 — Custom Networks & Compose Essentials

Isolating the tiers of a multi-service app, env-variable precedence, and the Compose commands worth knowing

Project: `step5-custom-networks` / `step5-practice` • Next.js → Express → Redis • Updated 29 June 2026

What this guide is. Part 1 is a beginner-friendly walkthrough of *custom* Docker networks — giving each tier its own network so the frontend can't reach the database directly. Part 2 adds two essentials we covered alongside it: **environment-variable precedence** and the **Compose commands** you should be able to explain (including `exec` vs `run`), with interview-ready phrasing.

UPDATED Now includes **Part 2 — Compose Essentials** (env precedence + commands).

📋 **Contents**

PART 1 — CUSTOM NETWORKS

1. Why custom networks? (the problem)

By default, Compose puts **every service on one shared network** (`<project>_default`). That's convenient — but it means *any* container can reach *any* other. In particular, your **frontend can talk directly to Redis**, even though it never should.

You proved this on the default network: from inside the frontend, a Redis `PING` came straight back:

```
# on the DEFAULT network (everything together)
$ docker compose exec frontend getent hosts redis
172.18.0.2   redis            # ← frontend can SEE redis
# ...and a TCP PING returned: +PONG   ← frontend can TALK to redis
```

> 🚨 **The risk:** the database should only be reachable by the backend. On a flat network there's no wall — if the frontend (the most exposed tier) were ever compromised, it has a direct line to your data.

2. The goal: isolate the tiers

A three-tier app has a natural "chain of trust":

```
browser ⟶ frontend ⟶ backend ⟶ redis
(public)    (UI)       (logic)   (data)
```

Each tier should only talk to its **immediate neighbour**:

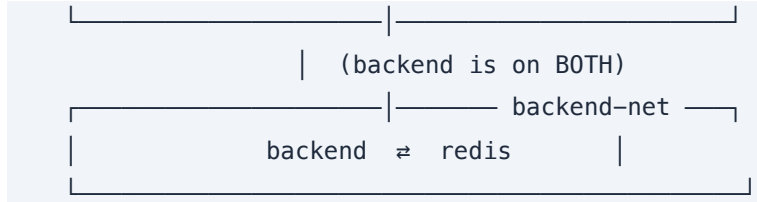
- frontend → backend ✅ (needed)
- backend → redis ✅ (needed)
- frontend → redis ❌ (should be blocked)

Custom networks let us enforce exactly that — a principle called **defense in depth**: even inside your app, services can't reach things they don't need.

3. The two-network design

We create **two** networks and make the backend the bridge between them:

```
┌──────────────── frontend-net ────────────────┐
│   frontend  ⇄  backend            │
```



Service	Network(s)	Can reach
frontend	`frontend-net`	backend ✅ · redis ❌
backend	`frontend-net` + `backend-net`	frontend ✅ · redis ✅
redis	`backend-net`	backend ✅ (only)

4. The Compose syntax

Two parts: (a) a `networks:` key on each service saying which networks it joins, and (b) a top-level `networks:` block that declares the networks exist.

```
services:
  frontend:
    build: ./frontend
    ports: ["3000:3000"]
    networks:
      - frontend-net           # frontend joins ONE network

  backend:
    build: ./backend
    networks:
      - frontend-net           # backend joins BOTH
      - backend-net

  redis:
    image: redis:7
    networks:
      - backend-net            # redis joins ONE network

# Declare the networks (Docker creates them on `up`)
networks:
  frontend-net:
  backend-net:
```

Two halves to remember: the `networks:` under each service = "which networks this service plugs into." The `networks:` at the bottom = "these networks exist." You need both.

5. Who joins which network (and why)

💡 **The rule:** a container can only reach others on a network it **shares**. So each service joins exactly the networks it needs — no more.

- **frontend** only needs the backend → 1 network (`frontend-net`).
- **backend** needs the frontend AND redis → 2 networks (both).
- **redis** only needs the backend → 1 network (`backend-net`).

The backend is the only service that needs to reach *both* tiers, so it's the only one on both networks — the **bridge**.

6. How the frontend still reaches the backend

A natural question: the frontend isn't on `backend-net` — so how does it talk to the backend at all?

Answer: through the network they SHARE — `frontend-net`. Both the frontend and the backend are members of `frontend-net`, and that one shared network is all they need.

> 💡 **Key rule:** two containers can communicate over *any* network they **both** belong to. They only need **one** network in common. The backend's extra membership in `backend-net` is a separate, private line to Redis — invisible to the frontend.

```
frontend ──(frontend-net, shared)──▶ backend ──(backend-net, private)──▶ redis
```

7. The big reveal: one container, two IPs

When you inspected your two networks on `step5-practice`, the proof was right there. The **backend appeared in BOTH networks — with a DIFFERENT IP in each:**

Network	Subnet	Members & their IPs
`backend-net`	172.18.0.0/16	redis → `172.18.0.2` **backend → 172.18.0.3**
`frontend-net`	172.19.0.0/16	**backend → 172.19.0.2** frontend → `172.19.0.3`

> **Look at the backend:** `172.18.0.3` on backend-net AND `172.19.0.2` on frontend-net. **A container on two networks gets a separate address on each.** That's literally what makes it the bridge — one foot (and one IP) in each network.

Meanwhile **frontend** only has `172.19.x.x` and **redis** only has `172.18.x.x` — they have no address on each other's network at all.

8. Two separate subnets

Notice the two networks have **different subnets**:

```
frontend-net : 172.19.0.0/16   (addresses 172.19.x.x)
backend-net  : 172.18.0.0/16   (addresses 172.18.x.x)
```

Docker gives **each network its own separate address space**. These aren't sub-ranges of one another — they're two independent "neighbourhoods." A device in one has no address in the other.

> 🏠 **Analogy:** `frontend-net` and `backend-net` are two different neighbourhoods. frontend lives only in one, redis only in the other. The backend owns a house in *both* (a different street address in each), so it can carry messages between them. Frontend and redis live in different neighbourhoods with no road connecting them.

9. Proving the isolation (hands-on)

From inside the frontend container, check whether each name resolves. `getent hosts <name>` returns an IP if it resolves, and **nothing** if it can't.

```
$ docker compose exec frontend sh

# backend IS reachable (shared frontend-net)
# getent hosts backend
172.19.0.2   backend          ✅ resolves

# redis is NOT reachable (no shared network)
# getent hosts redis
#                              ⬜ empty = isolated!
# echo $?
2                              ← exit code 2 = name not found
```

💡 **The empty output IS the success.** Seeing `backend` resolve but `redis` come back empty proves the isolation: the frontend can talk to the backend but is completely walled off from Redis.

⚠️ **Watch the spelling.** It's `getent hosts` (plural — same word as the `/etc/hosts` file). `host` (singular) gives "Unknown database"; a misspelling like `gentent` gives "not found" — neither is the real isolation result.

10. Default vs custom — side by side

	Default network (step4)	Custom networks (step5)
Networks	one (`<project>_default`)	two (`frontend-net`, `backend-net`)
frontend → backend	✅ works	✅ works
backend → redis	✅ works	✅ works
frontend → redis	⚠️ works (no isolation)	✅ blocked (`ENOTFOUND`)
`getent hosts redis` from frontend	`172.18.0.2`	(empty, exit 2)

Same app, same code — the **only** difference is the network setup in `docker-compose.yml`.

11. Command gotchas you hit

Command	Problem	Fix
`docker network inspect`	"requires at least 1 argument" — no network named	name it: `docker network inspect <name>`
`docker inspect network <name>`	tried to inspect a thing literally called "network"	`docker network inspect <name>` (network is a subcommand)
`getent host redis`	"Unknown database: host" — wrong database name	`getent hosts redis` (plural)
`gentent hosts redis`	"not found" — typo in the command	`getent hosts redis`

> **Pattern:** commands that act on a specific thing (`inspect`, `exec`, `logs`) need you to *name* that thing. "additional properties / requires N arguments / unknown" errors almost always mean a typo or a missing target.

12. Bonus: `internal: true`

By default a custom network can still reach the **internet** (via the gateway). For a database, you often don't want that. Marking a network `internal` seals it off from the outside world entirely:

```
networks:
  frontend-net:
  backend-net:
    internal: true        # backend-net can't reach the internet
```

Now Redis (on `backend-net`) can talk to the backend but has **no route to the internet** — an extra layer of lockdown. The backend still reaches the outside via `frontend-net`. Optional, but a nice real-world touch.

PART 2 — COMPOSE ESSENTIALS

13. Environment-variable precedence

You now use **both** `env_file:` and `environment:`. So what happens when the *same* variable is set in both? We tested it — and **`environment:` won**:

```
env_file:     GREETING=from_env_FILE
environment:  GREETING=from_ENVIRONMENT_block
result →      from_ENVIRONMENT_block        ✅ environment: wins
```

> 💡 **The mental model:** treat `env_file:` as the *defaults*, and `environment:` as *overrides* written right in the compose file. The more specific source wins.

The full precedence order (highest wins)

#	Source	Notes
1	`docker compose run -e VAR=…`	one-off command-line override — beats everything
2	your **shell** environment (via `${VAR}`)	what you export in the terminal
3	**`environment:`** in compose	← won the demo
4	**`env_file:`** in compose	the "defaults" layer
5	**`ENV`** in the Dockerfile	baked into the image; weakest

It runs **least-specific → most-specific**: the Dockerfile default is easiest to override; a value you type at runtime is hardest. Each layer overrides the broader one beneath it.

> ⚠️ **Reminder:** changing any env value needs a **recreate** (`docker compose up -d`), not just a `restart` — env vars are set when a container starts.

14. Compose commands you should know

The commands that matter day-to-day, with the distinctions interviewers probe.

`exec` vs `run` — the classic pairing

The difference is **which container your command runs in**:

```
exec  →  jumps INTO a container that's ALREADY running
run   →  creates a BRAND-NEW container just for your command
```

	`exec`	`run`
Container used	the **existing** running one	a **new** one each time
Service must be up?	Yes	No (creates its own)
State it sees	the live container's current state	**fresh** from the image
When command exits	container **keeps running**	new container **stops** (remove with `--rm`)
Typical use	debug/inspect a live service	run a task: migrate, seed, test
Published ports	uses the running container's	**not** mapped by default

```
# exec — into the live container (must be running)
docker compose exec backend sh
docker compose exec redis redis-cli

# run — fresh throwaway container for a task
docker compose run --rm backend npm run migrate
```

🎤 **Interview answer:** " `exec` runs inside a container that's already up — I use it to open a shell or run a command against a live service. `run` spins up a fresh one-off container from the service's image to do a task, like `docker compose run --rm backend npm run migrate`, and it's discarded after. Rule of thumb: `exec` for a live container, `run` for a throwaway task."

🚌 **Analogy:** `exec` = boarding a bus that's already driving (hop on, do something, hop off — it keeps going). `run` = renting an identical car for one errand, then returning it (the original bus never notices).

⚠️ **Two `run` nuances:** add `--rm` or the stopped container lingers; and `run` doesn't publish the service's ports by default (use `--service-ports` if needed).

build vs up --build

`docker compose build` builds the images from the Dockerfiles (for services with a `build:` key) **without starting** them. `up --build` does build + run together.

> 🎤 **Interview answer:** " `build` just bakes the images and stops; `up --build` rebuilds then runs. I use plain `build` in CI when I only want the image."

up/down vs stop/start

Command	What it does
`up`	**creates** + starts containers (and the network)
`down`	stops + **removes** containers and network
`stop`	**halts** running containers but **keeps** them
`start`	runs those same **existing** containers again

> 🎤 **Interview answer:** " `stop` / `start` just pause and resume *existing* containers and preserve their state; `down` / `up` actually *remove* and *recreate* them. Add `-v` to `down` to also delete volumes."

`--scale` and `pull`

```
docker compose up --scale backend=3    # run 3 copies of backend
docker compose pull                     # download images (no build/run)
```

`--scale` runs multiple replicas; Docker's internal DNS round-robins across them (only works if the service doesn't publish a fixed host port). `pull` just downloads pre-built images like `redis:7`.

> 🎤 **How to answer ANY command question (3-part structure):** (1) what it does, (2) when you'd use it, (3) contrast with a sibling command. The contrast is what shows you understand the system, not just the word.

15. Commands cheat-sheet

```
# ── Networks ──────────────────────────────────────────
docker network ls                          # list networks
docker network inspect <name>               # members, subnet, gateway (NAME require
docker compose exec frontend getent hosts backend   # prove DNS resolution
docker network prune                       # remove unused networks

# ── Lifecycle ─────────────────────────────────────────
docker compose up --build                  # build images + start
docker compose up -d                       # start in background (applies env change
docker compose build                       # build images only (no start)
docker compose pull                        # download images only
docker compose stop / start                # halt / resume EXISTING containers
docker compose down [-v]                   # remove containers+network (+volumes wit

# ── Running commands ──────────────────────────────────
docker compose exec backend sh             # shell in a LIVE container
docker compose run --rm backend npm run migrate   # NEW throwaway container for a ta
docker compose up --scale backend=3        # run 3 replicas
```

16. Key takeaways & glossary

- The default network puts everything together → **no isolation**; custom networks let each tier reach only its neighbour.
- A container only needs **one shared network** with another to reach it; one on multiple networks gets a **separate IP per network** (the bridge).
- frontend and redis share no network → `getent hosts redis` returns nothing = isolated.
- When a variable is set twice, **`environment:` beats `env_file:`** (and CLI/shell beat both; Dockerfile `ENV` is weakest).
- **`exec`** = command in a live container; **`run`** = fresh throwaway container for a task.

Term	Meaning
Custom network	a network you define yourself (vs the auto-created default)
Tier isolation	limiting each service to only the services it must reach
Bridge service	a container on two networks that links them (your backend)
Defense in depth	multiple layers of restriction, even inside your own app
`internal: true`	seals a network off from the internet
Precedence	which env source wins when a variable is set in several
`exec`	run a command inside an already-running container
`run`	create a new one-off container to run a task, then discard
`--scale`	run multiple replicas of a service
`getent hosts`	look up a name→IP; empty result = doesn't resolve
